

JavaScript

Avec l'arrivée de Java (1995) la notion d'Applet fait son apparition. Ils sont (pour l'époque) une manière **portable** d'exécuter du code côté client.

Problème : Java est trop lourd à l'époque (c'est un vrai langage, il fait peur aux créateurs de site, les performances sont médiocres, ...).

1995 : Brendan Eich (Netscape) crée Javascript en 10 jours. Il emprunte de la syntaxe à Java/C, et Netscape Navigator 2.0 embarque un interpréteur Javascript en standard

Le langage est rapidement adopté, mais chaque navigateur implémente sa propre variante. Le langage lui-même est **standardisé** en 1996 (ECMAScript, standardisé par l'ECMA).

2009 : Standardisation ISO de ECMAScript 5 🇺🇸 (2011 pour la version 5.1)

2015 : Standardisation ISO de ECMAScript 6 🇺🇸

2016 : Standardisation ISO de ECMAScript 7 🇺🇸

2017 : Standardisation ISO de ECMAScript 8 🇺🇸

2018 : Standardisation ISO de ECMAScript 9 🇺🇸

## Web Dynamique ?

Le modèle du Web présenté précédemment est **statique**. Les documents sont stockés sous forme de fichiers physiques, sur le disque dur d'un serveur.

Très tôt on a souhaité générer **dynamiquement** le contenu d'une page.

1993 : invention des scripts CGI qui permettent au serveur de récupérer les paramètres d'une requête HTTP et de générer du HTML en réponse.

La programmation Web **côté serveur** évolue ensuite (apparition de PHP en 1994, puis possibilité ensuite de programmer le côté serveur dans des langages plus robustes, comme Java, ...)

Un problème subsiste : le manque d'interactivité. En effet, on est contraint par le modèle :

formulaire HTML → envoi au serveur → calcul de la réponse → retour au client → rechargement de page. Problème d'interactivité (latence réseau, rendu graphique d'une nouvelle page, ...).

# JavaScript

---

- Quid ?
  - Langage de programmation lié à HTML.
  - Code JavaScript intégré aux pages HTML.
  - Code interprété par le navigateur client (interprétation dépendante du type et de la version de navigateur).
  - ≠ code PHP (interprété du côté serveur).
- Java Script est un langage
  - à prototype (par opposition à un langage basé sur les classes et sous classes pour réaliser l'héritage) Tout objet JavaScript est doté d'une propriété prototype, qui représente le modèle sur lequel l'objet en question se base.



# Comment exécute-t-on du Javascript ?



◆ Côté client : le code javascript est exécuté par le navigateur Web. Il doit donc être référencé dans une page HTML :

- ◆ Soit en utilisant l'attribut **src** d'une balise **script**
- ◆ Soit en mettant le code directement dans une balise **script**

```
<html>
...
<script type="text/javascript" src="toto.js"></script>
...
<script type="text/javascript">
  alert("Hello, World!");
</script>
</html>
```

◆ Côté serveur : on peut maintenant utiliser Javascript comme un langage généraliste grâce à l'interpréteur **Node.js**

- les éléments situés dans l'en-tête se comportent comme des déclarations, ils ne s'exécutent pas directement
- les éléments situés dans le corps s'exécutent au fur et à mesure du chargement de la page

## Description du langage

Javascript est un langage :

- ◆ **Dynamique** (tout est fait à l'exécution)
- ◆ En particulier **dynamiquement typé** (donc pas typé)
- ◆ **Impératif** (effets de bords, boucles **for**, notion d'instruction, ...)
- ◆ **Fonctionnel** (les fonctions sont des objets de première classe que l'on va manipuler à haute dose)
- ◆ **Objet** (mais sans notion de classe, ce qui rend la chose ~~méridique~~ amusante)
- ◆ **Interprété**, avec une compilation **à la volée** (JIT). Les navigateurs Web modernes ont des performances incroyables (possibilité de faire des jeux 3D par exemple)

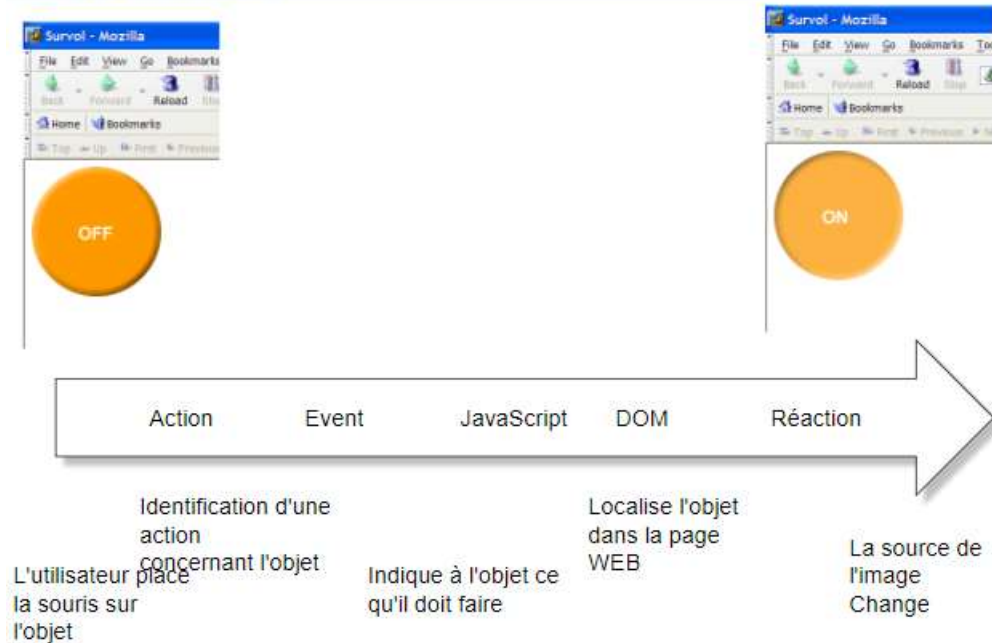
## Environnement de développement

- ◆ Pour les premiers cours, on utilisera les navigateur Chrome<sup>TM</sup> ou Firefox<sup>TM</sup>.
- ◆ Il est recommandé d'utiliser le même navigateur pour s'abstraire dans un premier temps des problèmes de compatibilité
- ◆ On peut utiliser un éditeur de texte simple (Eclipse est à proscrire, le support Javascript est notoirement mauvais). Visual Studio Code (Microsoft) est un bon éditeur de code Javascript.
- ◆ On utilisera la console de débogage Javascript du navigateur (Ctrl-Shift-J)

## Les objets

- Un objet est un élément nommé ayant des
  - **Propriétés** : paramètres que vous vérifier et modifier.
  - **Méthodes** : actions que l'objet est capable d'effectuer.
  - **Événements** : choses qui arrivent à l'objet, auxquelles celui-ci peut répondre automatiquement par une action.

## Les événements



## JavaScript

### • Intérêts de JavaScript ?

- Supporté (par défaut) par les principaux navigateurs, ne nécessite pas de plug-in particulier.
- Accès aux objets contenus dans un document HTML (+ possibilité de les manipuler relativement facilement).
- Possibilité de mettre en place des animations sans l'inconvénient des longs temps de chargement nécessités par les données multimédia.
- Langage relativement sécurisé : il est impossible de lire ou d'écrire sur le disque client (impossibilité de récupérer un virus par ce biais).

### • Arrivée d'AJAX

### • Difficultés d'utilisation de JavaScript ?

- Comme pour HTML, il n'y a pas de standard pour l'accès aux différents objets d'un document (dépendant du navigateur).
- Si le script ne fonctionne pas, la page est, le plus souvent, inutilisable. *Attention au menu non visible !*
- Les utilisateurs peuvent empêcher l'exécution de code JavaScript, souvent en raison des erreurs générées par les scripts, ou encore en raison de la nature de l'interaction (apparition de nouvelles fenêtres, ...).
- Lenteur d'exécution des scripts, ainsi que pour les scripts complexes, un certain délai avant le démarrage

## 4 types de base

**entier** : 127 (base 10), 0755 (base 8), 0xFA15 (base 16)

**flottant** : 0.123, -0.4e5, .67E-89

**booléen** : true, false

**chaîne de caractères** : "chaine" ou 'chaine'

Il n'y a pas de type entier, uniquement des **numbers** qui sont flottants IEEE-754 double précision (64 bits : 53 bits pour la mantisse, 11 bits pour l'exposant, 1 bit de signe).

**Entiers base 10** : 10, 3444, -25, 42, ...

**Flottants** : 1.3, 0.99, 00.34e102, -2313.2313E-23, ...

**Entiers base 8** : 0755, -01234567, ...

**Entiers base 16** : 0x12b, -0xb00b5, 0xd34db33f, ...

Le standard garantit que tous les entiers 32bits sont représentables exactement (sans arrondi). On peut écrire des entiers plus grands que  $2^{31}-1$  mais au bout d'un moment on a une perte de précision.

# Typage et Variable

## Pas de déclaration des variables

```
nbr = 10;  
f1 = 3.141;  
str1 = "L'étoile";  
str2 = 'brille';  
lien = '<a href="index.htm">Home</a>';
```

## Portée des variables

- locale (uniquement dans le script ou la fonction)

```
var vloc = 0 ;
```

- globale (en tout point du document)

```
vglob = 0 ;
```

Opérateurs arithmétiques :

|              |                                         |
|--------------|-----------------------------------------|
| - :          | « Moins » unaire                        |
| +, -, *, % : | addition, soustraction, produit, modulo |
| / :          | Division (flottante)                    |

## Booléens (boolean)

**true/false :**

vrai/faux

Opérateurs logiques :

! :

négation (unaire)

&&, || :

« et » logique, « ou » logique

## Variables, affectations

- ◆ Un nom de variable commence toujours par une lettre (majuscule ou minuscule), \$ ou \_ et se poursuit par un de ces caractères ou un chiffre.
- ◆ Les variables sont définies au moyen de **const** ou **let**

Exemples :

```
const $foo = 123;  
let bar = 1323e99;  
let _toto = bar;  
bar = 44;      //mise à jour  
$foo = 32;     //erreur, $foo est const
```

*Attention* on peut définir une variable sans l'avoir déclarée, et ça « marche » mais ça ne fait pas ce que l'on pense.

*Attention* le mot clé **var** existe, mais il est à proscrire (explication plus tard).

## Chaînes de caractères (string)



Encodées en UTF-16 (ou UCS-2), délimitées par des « ' » ou « " »

Opérations sur les chaînes :

- foo[10] :** accès au 11<sup>ème</sup> caractère, renvoyé sous la forme d'un chaîne contenant ce caractère
- pas de mise à jour :** les chaînes sont immuables
- +** : concaténation
- s.length :** longueur
- s.concat("23") :** concaténation (bis)

Un grand nombre de méthodes sont disponibles, on les verra prochainement (expressions régulières, recherche, remplacement, ...)

## null et undefined



*null* est une *constante* spéciale, de type *object*. Elle permet d'initialiser les variables comme en Java.

*undefined* est une *constante* spéciale, de type *undefined*. Elle correspond à la valeur d'une variable non initialisée ou d'une propriété non existante pour un objet.

# Comparaisons

---

## Opérateurs de comparaisons

| Opérateur              | Description                                      |
|------------------------|--------------------------------------------------|
| <code>a == b</code>    | Égal, après conversion de type                   |
| <code>a != b</code>    | Différent, après conversion de type              |
| <code>a === b</code>   | Égal et de même type                             |
| <code>a !== b</code>   | Différent ou de type différent                   |
| <code>a &lt; b</code>  | Strictement plus petit, après conversion de type |
| <code>a &gt; b</code>  | Strictement plus grand, après conversion de type |
| <code>a &lt;= b</code> | Plus petit, après conversion de type             |
| <code>a &gt;= b</code> | Plus grand, après conversion de type             |

# Objets

---

La structure de données de base est l'objet

```
{ }           //Un objet vide
{ x : 1, y : 2 } //Un objet avec deux champs x et y.
o.x           //Accès à un champ
o['x']        //Syntaxe alternative
o.z = 3;      //rajoute le champ z à l'objet o !
```

En javascript, tout est objet

```
"123".concat("456") //renvoie la chaîne "123456"
3.14.toString()     //renvoie la chaîne "3.14"
```

## Structures de contrôle : conditionnelle

```
if (c) {  
    // cas then  
} else {  
    // cas else  
}
```

Les *parenthèses* autour de la condition *c* sont obligatoires. La branche *else { \_ }* est optionnelle. Les accolades sont optionnelles pour les blocs d'une seule instruction

## switch/case

```
switch ( e ) {  
    case c1:  
        bloc1  
  
    case c2:  
        bloc2  
  
    ...  
    default:  
        blocdefault  
}
```

- ♦ l'expression *e* est évaluée et sa valeur comparée tour à tour aux *constantes* *c<sub>i</sub>*
- ♦ s'il y a égalité, le *bloc* correspondant est évalué. En fin de bloc, on exécute **break** pour sortir du **switch**, sinon le *bloc suivant est exécuté*.
- ♦ si aucune égalité et présence d'un label **default**, *bloc<sub>default</sub>* est exécuté, sinon on sort du **switch**
- ♦ les constantes peuvent être des entiers, booléens ou chaînes de caractères.

## Boucles



```
while ( c ) {  
    //corps de la boucle while  
}  
  
do {  
    //corps de la boucle do  
} while ( c );  
  
for(init ; test ; incr) {  
    //corps de la boucle for  
}
```

Il existe des constructions plus modernes (itérateurs de tableaux, boucles « foreach » ) qui seront détaillées au fur et à mesure du cours.

## break et continue

|                   |                                 |
|-------------------|---------------------------------|
| <b>break :</b>    | sort de la boucle immédiatement |
| <b>continue :</b> | reprend à l'itération suivante  |

## Exceptions



Syntaxe similaire à Java/C++ :

```
try{  
    ...  
} catch (ex) { /* faire qqchose avec ex */ }
```

On peut lever une exception avec **throw (e)**, où **e** peut être n'importe quelle valeur.  
À l'inverse de Java, il n'y a pas de multi-catch, car il n'y a pas de type pour différencier les objets.

# Fonctions

## Définition

```
function nomfonction(param1, ..., paramN) {  
    // code JavaScript  
    return expression ;  
}
```

## Appel

```
nomVariable = nomfonction(exp1, ..., expN);
```

passage des paramètres par valeur

On peut définir des fonctions globales :

```
function f(x1, ..., xn) {  
    // instructions  
};
```

On utilise le mot clé **return** pour renvoyer un résultat (ou quitter la fonction sans rien renvoyer)

On peut aussi créer des fonctions « inline » :

```
let z = 1 + (function (x, y) { return x * y; })(2,3);  
// x contient 7
```

On dispose donc de la syntaxe alternative pour la déclaration de fonction :

```
let f = function (z) { return x+1; };
```

## Fonctions anonymes



Les fonctions anonymes étant souvent utilisées, ES6 introduit une notation allégée :

```
let f = (x, y) => x + y; // le corps consiste en une seule  
                        // expression dont la valeur est renvoyée
```

```
let g = (x, y, z) => { // fonction dont le corps est un bloc  
    let u = x + y + z;  
    let v = x - y - z;  
    return u * v;  
};
```

## Fonctions et objets



En première approximation, « les méthodes » ne sont que des fonctions stockées dans les champs d'un objet :

```
let obj = { x : 1, y : 1 }; // objet
obj.move = function (i, j) {
  obj.x += i;
  obj.y += j;
};

obj.move(2,3);
```

On verra que c'est beaucoup plus ~~crade~~ subtil que ça.

## Objet global document



L'objet global **document** représente le document HTML. Il implémente l'interface DOM et on peut donc le parcourir comme un arbre (propriétés **firstChild**, **parent**, **nextSibling** ...).

La méthode **document.getElementById("foo")** permet de récupérer un objet représentant l'élément HTML de la page ayant l'attribut **id** valant **"foo"** (**null** si cet élément n'existe pas)

# Éléments HTML



Les éléments HTML (`document` ou les objets renvoyés par `getElementById`) implémentent l'interface DOM du W3C (Document Object Model, une spécification pour faire correspondre des concepts HTML sur des langages Objets). Les méthodes qui vont nous intéresser pour le TP :

`foo.addEventListener("event", f)` : Exécute la fonction `f` quand l'évènement "`event`" se produit sur l'élément `foo` (ou un de ses descendants).

`foo.innerHTML = "<b>Yo !</b>"` : Remplace tout le contenu de l'élément `foo` par le fragment de document contenu dans la chaîne de caractère.

`foo.value` : Permet de modifier ou récupérer la valeur de certains éléments (en particulier les zones de saisies de texte).

`foo.style` : Accès au style CSS de l'objet, représenté comme un objet Javascript

## Évènements



Les navigateurs Web se programment de manière événementielle : on attache des fonctions (*event handlers*) à des éléments. Quand un certain évènement se produit, la fonction est appelée :

```
//récupération de l'élément
let divToto = document.getElementById("toto");
//On suppose qu'il existe et on ne teste pas null

toto.addEventListener("click", (e) => {
  if (divToto.style.background != "") {
    //la feuille de style reprend le dessus
    divToto.style.background = "";
  } else {
    // on modifie le style par défaut pour l'élément toto
    divToto.style.background = "pink";
  }
});
```

Le paramètre `e` que prend la fonction permet de récupérer des informations sur l'évènement (coordonnées du pointeur, touche pressée au clavier, ...)

# Débuggage : objet console



L'objet **console** possède une méthode **log** permettant d'afficher un objet dans la console du navigateur. C'est un affichage **riche** qui permet d'explorer un objet affiché (démonstration).

## Lire/Écrire

### `prompt()`

Ouvre une boîte de dialogue avec une zone saisie et 2 boutons OK et Annuler, retourne l'information lue

### `confirm()`

Ouvre une boîte de dialogue avec 2 boutons OK et Annuler, retourne un booléen

### `alert()`

Permet d'écrire un message dans une fenêtre

## Lire/Écrire

### Exemple

```
<html>
<head>
  <title>Utilisation de prompt() et d>alert()</title>
</head>
<body>
  <script type = "text/javascript">
    //
      annee = prompt('En quelle année sommes-nous ? ', 2000);
      alert('Vous avez répondu : ' + annee);
    //]]&gt;
  &lt;/script&gt;
&lt;/body&gt;
&lt;/html&gt;</pre></div>
```

## Ouverture d'une fenêtre

`open(url, name, options)`

Permet d'ouvrir une fenêtre et d'écrire dedans

### Exemple

```
<script type = "text/javascript">
  //<![CDATA[
    fenetre = open('', '', 'height=50, width=300, status=yes')
    fenetre.document.write('<html>');
    fenetre.document.write('<head>');
    fenetre.document.write('<title>' + 'Titre fenetre' + '</ti
    fenetre.document.write('</head>');
    fenetre.document.write('<body>');
    fenetre.document.write('Texte dans la fenetre');
    fenetre.document.write('</body>');
    fenetre.document.write('</html>');
  //]]>
</script>
```

## Les options

`directories` : barre de liens

`menubar` : barre de menu

`status` : barre de statut

`location` : barre d'adresse

`scrollbars` : ascenseurs

`resizable` : redimensionnement par l'utilisateur

`height` : hauteur

`width` : largeur

`left` : position gauche

`top` : position haute

`fullscreen` : plein écran

## Les objets prédéfinis

### Les évènements

- `onclick` : un clic du bouton gauche de la souris sur une cible
- `onmouseover` : passage du pointeur de la souris sur une cible
- `onblur` : une perte de focus d'une cible
- `onfocus` : une activation d'une cible
- `onselect` : une sélection d'une cible
- `onchange` : une modification du contenu d'une cible
- `onsubmit` : une soumission d'un formulaire
- `onload` : un chargement d'une page
- `onunload` : la fermeture d'une fenêtre ou le chargement d'une page autre que la courante

### Exemple

```
<a href="" onclick="alert('Bonjour')">Cliquez</a>
```

## L'objet document

### Propriété

title

### Méthode

write

## L'objet String

Lorsqu'on définit une constante ou une variable chaîne de caractères, JavaScript crée d'une façon transparente une instance String

1 propriété : length

les balises HTML/XHTML ont leur équivalent en méthode

Liste (non exhaustive) des méthodes :

`bold()`, `italics()`, `fontcolor()`, `fontsize()`,  
`small()`, `big()`, `toUpperCase()`, `toLowerCase()`,  
`sub()`, `sup()`, `substring()`, `eval()`, `split()`, `replace()`

## L'objet Date

```
var maDate = new Date()
```

- `getFullYear()` : 2 chiffres
- `getFullYear()` : 4 chiffres
- `getMonth()` : 0 – 11
- `getDate()` : 1 – 31
- `getDay()` : 0 – 6 (dimanche – samedi)
- `getHours()` : 0 – 23
- `getMinutes` : 0 – 59
- `getSeconds()` : 0 – 59

# L'objet RegExp

L' **RegExp** objet est utilisé pour faire correspondre le texte avec un motif.

```
var re = new RegExp(motif, option)
```

- règles classiques pour la formation des motifs
- options possibles : "g", "i", "gi", ""

Les méthodes

- `test()` : booléen
- `exec()` : retourne la première occurrence
- `match()` : s'applique à un objet de type String

## L'objet Math

Propriétés : `Math.PI` et `Math.E`

Méthodes :

```
atan(), acos(), asin(), tan(), cos(), sin(),  
abs(), exp(), log(), max(), min(), pow(),  
round(), sqrt(), floor(), random()
```

# Les tableaux

```
var T = new Array()
```

- `new Array(n)` : Initialise un tableau de taille **n** (indiqué de 0 à **n-1**) où toutes les cases valent **undefined**
- `.length` : renvoie la longueur du tableau
- `.toString()` : applique `.toString()` à chaque élément concatène chaque résultat avec des "," intermédiaires.
- `.push(e)` : ajoute un élément en fin de tableau
- `.pop()` : retire et renvoie le dernier élément du tableau. **undefined** si le tableau est vide
- `.shift()` : retire et renvoie le premier élément du tableau. **undefined** si le tableau est vide
- `.unshift(e)` : ajoute un élément au début du tableau
- `.splice(i, n, e1, ..., ek)` : à partir de l'indice **i**, efface les éléments **i** à **i+n-1** et insère les éléments **e<sub>1</sub>, ..., e<sub>k</sub>**
- `.concat(t)` : Renvoie la concaténation du tableau et du tableau **t**
- `.sort(f)` : Trie le tableau en place. **f** est une fonction de comparaison (**f(a,b)** doit être négatif si **a<b**, nul si **a=b** et positif si **a>b**).

# Array, interface fonctionnelle



`.forEach(f)` : Applique la fonction **f** à tous les éléments du tableau qui ne valent pas **undefined**. **f** reçoit trois arguments (**v**, **i**, **t**) :

**v** : la valeur courante de la case visitée

**i** : l'indice courant (à partir de 0)

**t** : le tableau en entier

`.map(f)` : Applique la fonction **f** à tous les éléments du tableau qui ne valent pas **undefined** et renvoie le tableau des images. **f** reçoit trois arguments (**v**, **i**, **t**), comme pour **forEach**.

`.filter(f)` : Renvoie le tableau de tous les éléments pour lesquels **f** renvoie vrai. **f** reçoit trois arguments (**v**, **i**, **t**), comme pour **forEach**.

`.reduce(f, acc0)` : Renvoie l'aggrégat de tableau de tous les éléments par **f**. **f** reçoit quatre arguments (**acc**, **v**, **i**, **t**), où **acc** est la valeur courante de l'accumulateur et les autres sont comme pour **forEach**.

Si **acc<sub>0</sub>** est absent, exécute l'aggrégat entre le premier élément du tableau et le reste.

## Les tableaux classiques

```
var jours = new Array();

var jours = new Array("Lundi", "Mardi", "Mercredi", "Jeudi",
                      "Vendredi", "Samedi", "Dimanche");

jours[0]

jours.length
```

## Les tableaux associatifs

```
var tableau = new Array();

tableau["un"] = "La première chaîne";
tableau["deux"] = "La deuxième chaîne";
tableau["tnt"] = "Plein d'autres chaînes";

tableau["un"]

tableau.length
```

# Les méthodes de l'objet Array

```
var tableau3=tableau1.concat(tableau2);

var chaine=tableau.join(séparateur);

tableau.pop();

tableau.push(liste d'éléments);

tableau.reverse();

tableau.shift();
tableau.unshift(liste d'éléments);

tableau2=tableau1.slice(début, fin);

tableau.sort();

tableau.splice(début, longueur);
tableau.splice(début, longueur, liste d'éléments);
```

# L'objet form

## Attributs

- `name` : nom
- `action` : fichier
- `method` : get ou post
- `enctype` : encodage
- `target` : cadre cible (`_blank` pour une nouvelle page)

## Méthodes

- `submit()` : soumission
- `reset()` : remise à zéro

## Évènements

- `onSubmit()` : lors de la soumission
- `onReset()` : lors de la remise à zéro

# Les éléments de formulaires

## input text

```
<input type="text" id="motclef" value="Mot clef">
```

```
document.forms["monFormulaire"].elements["motcle"]
```

Les propriétés :

- value : valeur
- defaultValue : valeur par défaut
- form : objet formulaire
- maxLength : longueur maximale

Les méthodes :

- blur() : perte de focus
- focus() : prise de focus
- select() : donne le focus et sélectionne la zone de saisie

Les événements :

- onBlur : lors de la perte de focus
- onChange : lors d'un changement
- onFocus : lors de la prise de focus

## input button

Les propriétés :

- value : libellé

Les méthodes :

- click() : clic

Les événements :

- onClick : lors d'un clic

## select

Les propriétés :

- size : nombre de lignes
- options : tableau
  - ▶ value : valeur
  - ▶ text : libellé
  - ▶ defaultSelected : true of false
  - ▶ selected : true of false
- selectedIndex : indice de la ligne sélectionnée

## Exemple

```
<script type="text/javascript">
<!--
function basculer(orig, dest) {
    if (orig.options.selectedIndex >= 0) {
        var o = new Option(orig.options[orig.selectedIndex].text,
                            orig.options[orig.selectedIndex].value);
        dest.options[dest.options.length]=o;
        orig.options[orig.selectedIndex]=null;
    }
    else {
        alert("Aucune ligne sélectionnée");
    }
}

function monter(f) {
    var no=f.choix.selectedIndex;
    if (no<0) { return true; }
    if (no>=1) { inverser(f.choix, no, no-1); }
}

function baisser(f) {
    var no=f.choix.selectedIndex;
    if (no<0) { return true; }
    if (no<f.choix.options.length-1) { inverser(f.choix, no, no+1); }
}
```

```
function inverser(liste, ind1, ind2) {
    var valeur=liste.options[ind1].value;
    var texte=liste.options[ind1].text;
    liste.options[ind1].value=liste.options[ind2].value;
    liste.options[ind1].text=liste.options[ind2].text;
    liste.options[ind2].value=valeur;
    liste.options[ind2].text=texte;
    liste.selectedIndex=ind2;
}

function verifierListe(f) {
    if (f.choix.options.length == 0) {
        alert("Indiquez les raisons vous ayant poussé à créer votre site");
        return false;
    }
    for (var i=0; i<f.choix.options.length; i++) {
        f.choix.options[i].selected=true;
    }
    f.submit();
}
// -->
</script>
```

```
<form id="nomFormulaire" method="get" action="formu.html">
```

Double cliquez sur les raisons qui vous ont poussé; à; créer votre  
Internet, puis ordonnez les par ordre d'importance :

```
<table>
```

```
<tr>
```

```
<td>
```

Possibilités :<br>

```
<select id="raison" size="5" multiple="multiple"
```

```
    style="width:250px;" onClick="basculer(this, this.form.choix)">
```

```
    <option value="1">Apprendre le langage</option>
```

```
    <option value="2">Parler de ma passion</option>
```

```
    <option value="3">Créer une activité commerciale</option>
```

```
    <option value="4">Échanger avec le monde entier</option>
```

```
    <option value="5">Faire mon métier</option>
```

```
</select>
```

```
</td>
```

```
<td>
Vos choix :<br/>
<select id="choix" size="5" multiple="multiple"
        style="width:250px;" onDblClick="basculer(this, this.form.raison)">
</select>
</td>
<td>
<input type="button" value="Monter" onclick="monter(this.form)"><br/>
</td>
<td>
<input type="button" value="Baisser" onclick="baisser(this.form)"><br/>
</td>
</tr>
</table>
<p>
<input type="button" value="Valider" onclick="verifierListe(this.form)">
</p>
</form>
```

# Principes de la programmation orientée objet

Un général, un *langage orienté objet statiquement typé* propose une notion de *classe* et d'*objet*. Par exemple en Java :

```
class Point {  
    private int x;  
    private int y;  
  
    Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public void move(int i, int j) {  
        this.x += i;  
        this.y += j;  
    }  
    public int getX() { return this.x; }  
    public int getY() { return this.y; }  
}
```

Une *classe* définit un ensemble d'*objets* contenant un état interne (les attributs : *x*, *y*) ainsi que du code (les méthodes : *move*, ...) permettant de manipuler cet état. Un objet est l'*instance* d'une classe.

## Concepts objets

Les langages orientés objets exposent généralement plusieurs concepts :

- ◆ La notion de *constructeur* : une fonction ayant un statut spécial, qui est appelé pour *initialiser* l'état interne de l'objet à sa création.
- ◆ Une notion de *contrôle d'accès* aux attributs et méthodes.
- ◆ Un moyen de *référencer* l'objet dans lequel on se trouve (*this*).
- ◆ Plusieurs notions permettant de partager du code ou des données (*static*, héritage, ...)

# Identifiant **this**



L'identifiant **this** est similaire à celui de Java mais est tout le temps défini, avec des règles précises :

- ◆  $o.f(e_1, \dots, e_n)$  : **this** est initialisé à **o**
- ◆  $\text{new } f(e_1, \dots, e_n)$  : **this** est l'objet fraîchement créé.
- ◆  $f.\text{call}(o, e_1, \dots, e_n)$  : **this** est initialisé à **o**
- ◆  $f(e_1, \dots, e_n)$  :  **this** est initialisé à l'objet global (**window**).

```
ColoredPoint.prototype.getColor = function () {  
    console.log(this.color);           //accès à la couleur  
    let g = function () {  
        console.log(this.color);       //undefined car équivalent à  
                                        //window.color  
    };  
    g();  
    return this.color;  
};
```

## Définition d'objets simples

### Exemple

```
function Chien(nom, race, maitre) {  
    this.nom = nom;  
    this.race = race;  
    this.maitre = maitre;  
    this.print = printChien;  
}  
  
function printChien() {  
    document.write("Chien ", this.nom, " de race ", this.race,  
        " appartenant &agrave; ", this.maitre, "<br,  
}
```